I102 - Paradigmas de Programación

INTRODUCCIÓN A C++

2

Prof. Mariano Scaramal

Ingeniería en Inteligencia Artificial

Que conceptos de C++?

- Conceptos básicos de C++:
  - Tipos de variables,
  - Operadores,
  - Entrada y Salida de datos,
  - Sentencias,
  - Funciones,
  - Librerías básicas,
  - Comandos de preprocesador

Variables Primitivas

- Las variables primitivas son las variables con las que se crean las otras variables.
- En C++ estas son:
  - *bool:* True (1) o False (0)
  - *char:* carácter
  - *int:* entero
  - *float:* número real de precisión simple (o single precision)
  - *double:* número real de precisión doble (o double precision)
  - *void:* sin tipo, utilizado cuando una función no devuelve nada.

La variable tipo bool

- La misma puede definirse de distintas maneras:
- El tamaño de la variable bool es 1 byte (aunque solo se usa 1 bit).
- Al imprimirse, se mostrará como número entero, 1 para true y 0 para false. O bien “true” o “false”, mediante *boolalpha,*

```
#include <iostream>

int main(){
    bool a;  // Por default un bool se inicializa con false (cero)
    bool b = true;
    bool c{false};

    std::cout << "a es " << a << std::endl;  // a es 0
    std::cout << "b es " << b << std::endl; // b es 1
    std::cout << "c es " << c << std::endl;  // c es 0
    std::cout << "b es " << std::boolalpha << b << std::endl;
    // b es true
    std::cout << "Tamaño del bool  es " << sizeof(b) << std::endl;
    // Tamaño del bool es 1
    return 0;
}
```

La variable tipo char

- Esta variable contiene caracteres de la tabla ASCII, como ser:

  ‘a’, ‘F’, ‘@’, ‘#‘, etc

- Observe que estos caracteres usan las comillas simples.

- Es una variable de 1 byte con un rango entre -128 y 127.

```
#include <iostream>

int main(){
    char a;
    char b{'+'};                // + en char
    char c = 0b00101011;        // + en binario
    char d = 053;               // + en octal
    char e = 43;                // + en decimal
    char f = 0x2b;              // + en hexadecimal

    std::cout << "b es " << b << ", c es " << c << ", d es "
    << d << ", e es " << e << ", f es " << f << std::endl;
    std::cout << "El tamaño del char es : " << sizeof(char) <<
    std::endl;

    return 0;
}
```

b es +, c es +, d es +, e es +, f es +
El tamaño del char es: 1

La variable tipo char (cont'd)

- Existen los denominados "caracteres no gráficos", los cuales son utilizados para interpretar caracteres especiales, por ejemplo:

Carácter	Representación en ASCII	Valor en ASCII	Secuencia de Escape
Nueva línea	NL	10	\n
Tab horizontal	HT	9	\t
Retroceso	BS	8	\b
Retornar	CR	13	\r
Signo de pregunta	?	63	\?
Comillas simples	‘	39	\’
Comillas dobles	“	34	\”
Número octal	ooo	---	\ooo
Número hexadecimal	hhh	---	\xhhh
Carácter NULL	NUL	0	\0

- Tabla ASCII: es.wikipedia.org/wiki/ASCII

Las variables enteras

- La tabla presenta los tipos de enteros para Windows.
- Pueden ser con o sin signo, modificando su rango.

int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	2bytes	0 to 65,535
signed short int	2bytes	-32768 to 32767
long int	8bytes	-2,147,483,648 to 2,147,483,647
signed long int	8bytes	same as long int
unsigned long int	8bytes	0 to 4,294,967,295
long long int	8bytes	-(2^63) to (2^63)-1
unsigned long long int	8bytes	0 to 18,446,744,073,709,551,615

Las variables enteras (cont'd)

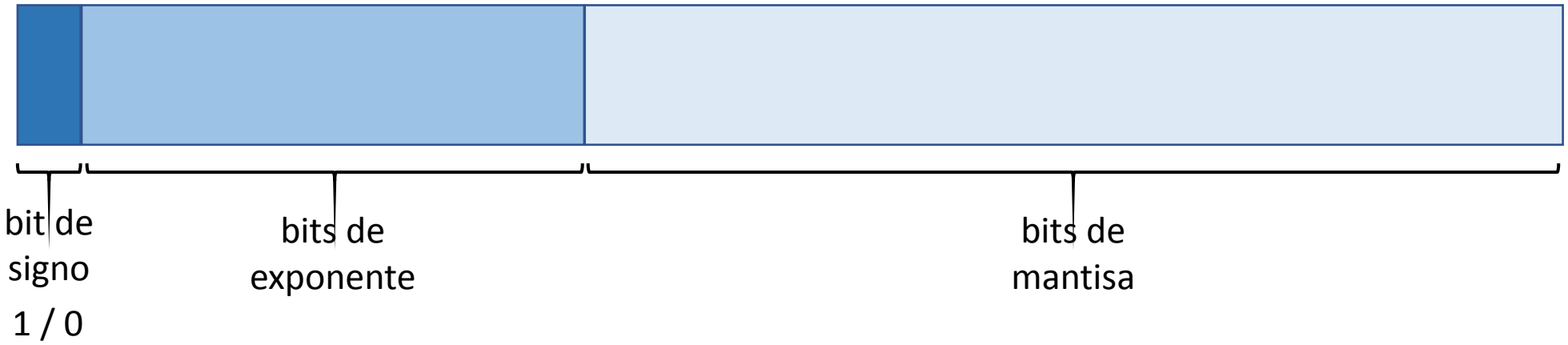
- Al igual que para char, existen distintas formas de definirlos.
- Si el rango es superado, una variable signada, hará overflow (ver variable "d").

```
#include <iostream>

int main(){
    unsigned int a = 85;          // decimal
    long long b{07747677};        // octal (usar cero antes del número)
    short c = {0xA};              // hexadecimal (usar 0x)
    short int d = +32768;

    std::cout << "b es " << b << ", c es " << c << ", d es " << d <<
    std::endl;
    std::cout << "El tamaño del long long es: " << sizeof(long
    long) << std::endl;

    return 0;
}
```

b es 2.084.799, c es 10, d es -32768
El tamaño del long long es: 8

Las variables de punto flotante

- Las variables de punto flotante tratan de interpretar los números reales.
- En C++ existen:
  - *float*: precisión simple con 32 bits, 8 bits en exponente y 23 bits en mantisa.
  - *double*: precisión doble con 64 bits, 11 bits en exponente y 52 bits en mantisa.
  - *long double*: precisión cuádruple con 128 bits, 15 en exponente y 112 en mantisa.



Las variables de punto flotante (cont'd)

Ejemplo con *long double* :

```
#include <iostream>
#include <iomanip>

using namespace std;

int main(){
    long double value = 3.14159265358979323846264338327 95;
    cout << "El valor es: " << setprecision(30) << value << endl;
    //          El valor es: 3.141592653589793115997963468 54
    return 0;
}
```

Si no se utiliza *setprecision* el resultado en pantalla sería simplemente *3.14159*, no obstante, al procesar el número se utilizará toda la información para proveer una aproximación al número.

Las variables de punto flotante (cont'd)

El cálculo del numero interpretado con un bits de signo, mantisa y exponente se realiza con la siguiente expresión:

$$x = (-1)^s \left[2^{\left(\sum_{i=0}^{e-1} E_i\, 2^i\right)-\left(2^{e-1}-1\right)}\right]\left[2^0 + \sum_{i=1}^{m} M_i\, 2^{-(m-i+1)}\right]$$

donde:

- S es el valor del bit de signo (1 o 0)
- e es el número de bits en el exponente (8, 11 o 15)
- E_i es el valor de cada bit en el exponente (1 o 0)
- m es el número de bits en la mantisa (23, 52 o 112)
- M_i es el valor de cada bit en la mantisa (1 o 0)

- Esto implica que hay ciertos números que no se podrán obtener exactamente.

Las variables de punto flotante (cont'd)

En el simple caso de utilizar una variable tipo float:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main(){
   float x = 1.0;
   x = x - 0.9;      // Es "x" igual a 0.1?
   cout << "El valor de x es: " << x << endl;
   cout << "El valor de x es: " << setprecision(30) << x << endl;

   x = x - 0.1;      // Es "x" igual a 0.0?
   if (x == 0.0)
      cout << "El valor de x es 0.0" << endl;
   else
      cout << "El valor de x no es 0.0" << endl;

   return 0;
}
```

El código imprime:

El valor de x es: 0.1
El valor de x es: 0.100000001490116119384765625
El valor de x no es 0.0

El tipo void y void*

- El compilador reconocerá la leyenda *void* como "*no tipo*".
- El tipo *void* suele utilizarse para crear funciones que no devuelven nada.
- En el caso de *void** esto se interpreta como un puntero a una variable que no fue definida o aún no tiene asignado un tipo.
- La familia de funciones *malloc/calloc/etc* devuelven *void**. Es decir un puntero a una variable que debe ser casteada o se le debe asignar un tipo:

*int *myArr = (int*) malloc(size*sizeof(int))*

- De esta manera, *malloc*, retorna la dirección a un espacio de memoria de un determinado tamaño (es este caso *size* veces el tamaño de un tipo *int*).
- El compilador necesita el tipo de variable para compilar el código y poder recorrer el array en tamaños tipo *int*.

El tipo void*

- El puntero a void permite castear un puntero a un tipo de variable.
- En el código, la función es utilizada para imprimir distintos tipos de variables.
- No es recomendable, luego se verán otras soluciones a este problema.

```
#include <iostream>

void print_var(void* data, size_t size) {
    if (size == sizeof(int)) {
        int* int_ptr = (int*)(data);
        std::cout << "Integer: " << *int_ptr << std::endl;
    } else if (size == sizeof(double)) {
        double* double_ptr = (double*)(data);
        std::cout << "Double: " << *double_ptr << std::endl;
    } else if (size == sizeof(char)) {
        char* char_ptr = (char*)(data);
        std::cout << "Character: " << *char_ptr << std::endl;
    } else {
        std::cout << "Tipo de dato no includio." << std::endl;
    }
}
```

```
int main() {
    int num = 42; double pi = 3.14159; char myChar = 'A';

    print_var(&num, sizeof(num));
    print_var(&pi, sizeof(pi));
    print_var(&myChar, sizeof(myChar));
    std::cout << std::endl;

    return 0;
}
```

El código imprime:

Integer: 42
Double: 3.14159
Character: A

Casting en C++

El lenguaje C++ realiza el cast utilizando:

static_cast<DataType>(Variable)

Este casting es más seguro dado que tiene verificación de tipo de dato.

```
#include <iostream>

int main() {
    int n = 10; // En hexa es: 0xA
    // Con el cast tradicional, compila y muestra 0xa
    void* ptr = (void*)n;  // Define posiciones de memoria con valores int
    std::cout << ptr << std::endl;

    // Con static_cast se tendrá: Invalid type conversion
    void* ptr2 = static_cast<void*>(n);   // No compila por esta linea

    return 0;
}
```

Entrada y Salida de Datos

La salida y entrada de información por pantalla se realiza mediante *cout* y *cin*, respectivamente.

Tanto *cout* como *cin* son objetos de las clases *std::istream* y *std::ostream*, las cuales se verán luego del primer parcial.

A continuación, los dos códigos imprimen *Hello, World!* en pantalla. El de la derecha, hace uso del namespace std. Esto evita repetir *std::* para indicar a quién pertenece el objeto *cout* y el manipulador de flujo *endl.*

El operador << es el operador de inserción, usado para insertar data en *cout*.

```
#include <iostream>

int main(){
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main(){
    cout << "Hello, World!" << endl;
    return 0;
}
```

Entrada y Salida de Datos (cont'd)

La entrada de información por parte del usuario se realiza mediante el objeto std::cin y el operador de extracción >>.

En el ejemplo, el nombre ingresado por el usuario se almacena en la variable name, que es de tipo std::string.

Cuando el usuario ingresa el nombre y presiona Enter, los caracteres escritos (incluido el carácter de salto de línea '\n') se almacenan en el buffer de entrada.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name, fullName;

    cout << "Ingrese su nombre: ";
    cin >> name;
    cout << "Ingrese su nombre completo: ";
    cin >> ws;  // Elimina los whitespaces del buffer
    getline(cin, fullName);

    cout << "Hola, " << name << "!" << endl;
    cout << "Tu nombre completo es: " << fullName << endl;

    return 0;
}
```

Entrada y Salida de Datos (cont'd)

El operador >> lee caracteres hasta encontrar un espacio en blanco y deja el carácter '\n' pendiente en el buffer.

Luego, cuando se utiliza std::getline, esta función lee hasta encontrar un salto de línea. Si ese '\n' ya estaba pendiente en el buffer, getline lo encuentra inmediatamente y devuelve una cadena vacía.

Para evitar este problema, se eliminan los caracteres de espacio en blanco pendientes utilizando **cin >> ws;**

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name, fullName;

    cout << "Ingrese su nombre: ";
    cin >> name;
    cout << "Ingrese su nombre completo: ";
    cin >> ws;  // Elimina los whitespaces del buffer
    getline(cin, fullName);

    cout << "Hola, " << name << "!" << endl;
    cout << "Tu nombre completo es: " << fullName << endl;

    return 0;
}
```

Entrada y Salida de Datos (cont'd)

Universidad de SanAndrés

Analizar diferencia entre ambos programas

```
#include <iostream>
#include <string>
using namespace std;

int main() {
   string name, fullName;

   cout << "Ingrese su nombre: ";
   cin >> name;
   cout << "Ingrese su nombre completo: ";
   cin >> ws;  // Elimina los whitespaces del buffer
   getline(cin, fullName);

   cout << "Hola, " << name << "!" << endl;
   cout << "Tu nombre completo es: " << fullName << endl;

   return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
   string name, fullName;

   cout << "Ingrese su nombre: ";
   cin >> name;
   cout << "Ingrese su nombre completo: ";
   getline(cin, fullName);

   cout << "Hola, " << name << "!" << endl;
   cout << "Tu nombre completo es: " << fullName << endl;

   return 0;
}
```

Entrada y Salida de Datos (cont'd)

Recordatorio:

- cin >> var:
  - Lee datos desde el buffer hasta el primer whitespace (espacio, tab o \n).
  - Ignora espacios en blanco iniciales automáticamente.
  - No permite leer cadenas con espacios.
  - Deja el '\n' en el buffer

- getline(cin, var):
  - Lee todos los caracteres hasta encontrar un '\n'.
  - Permite leer espacios dentro de la cadena.
  - No ignora automáticamente whitespace inicial.
  - Consume y descarta el '\n' del buffer.
  - Si hay un '\n' pendiente, puede devolver una cadena vacía.

Múltiples Entradas de Datos

- Anteriormente se vió que los datos pueden ser ingresados de a uno.
- Tener en cuenta que se aceptan distintos tipos de datos que pueden producir comportamiento no esperado si no se considera su tipo.

```
#include <iostream>
#include <typeinfo>

int main(){
    char c = 'a'; int i = 0; double d = 0.0;

    std::cout << "Ingrese un carácter, un entero y una variable tipo double (separados por espacios): ";
    std::cin >> c >> i >> d;
    std::cout << "Usted ingreso: " << c << ", " << i << ", " << "y " << d << std::endl;
    std::cout << "Tipo de dato de c: " << typeid(c).name() << std::endl;
    std::cout << "Tipo de dato de i: " <<  typeid(i).name() << std::endl;
    std::cout << "Tipo de dato de d: " << typeid(d).name() << std::endl;
    return 0;
}
```

Arrays

- Se trata de un conjunto de datos ordenados por un índice que posee una cantidad de elementos fija y no puede ser cambiada en tiempo de ejecución.
- Al igual que en C, el índice indica el desplazamiento de la posición inicial, es decir, el índice inicia en cero.
- Hay distintas formas de declarar un array:

```
int myArr[4] = {2, 3, 6, 1};
int myArr2[4] = {2};
int myArr3[] = {10, -8, 13, 17};
```

- En el 2do caso, se obtiene un array de 4 posiciones pero la primera es 2.
- En el 3er caso, el compilador determina que el array es de 4 posiciones.

Arrays (cont'd)

- Otra forma de declarar un array es definiendo su longitud pero sin hacerle una asignación:

```
char myStr[20];
```

- Para asignarle valores, se puede iterar sobre el mismo o se puede utilizar el objeto *cin:*

```
char myStr[20];
std::cin >> myStr;
```

- Es importante tener en cuenta que *cin*, por default, solo lee datos hasta el primer dato tipo *whitecharacter* (espacio, tab, enter, etc).

Matrices o Arrays Multidimensionales

- Son estructuras de tamaño fijo y su asignación es similar a la de los arrays.
- Tener en cuenta que los índices comienzan desde cero.

```
#include <iostream>

int main(){
    int myMtx[4][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}, {10, 11, 12}};
    std::cout << myMtx[2][0] << std::endl;      // Imprime el valor 7
    std::cout << myMtx[1][1] << std::endl;      // Imprime el valor 5
    std::cout << myMtx[3][2] << std::endl;      // Imprime el valor 12

    return 0;
}
```

- En C++ moderno es aconsejable el uso del container std::array.
- Los mismos se verán más adelante con el resto de los containers.

std::string

- A diferencia de C, C++ provee una variable tipo string (std::string es implementada con una clase).
- La implementación de std::string permite concatenar strings muy fácilmente

```
#include <iostream>
#include <string>

int main(){
    std::string s;
    std::string s1 = "Hello ";
    std::string s2 = "World";
    char c = '!';
    s = s1;
    s += s2;   // Lo mismo que hacer s = s1 + s2
    s += c;    // También permite concatenar chars
    std::cout << s << std::endl;

    return 0;
}
```

El código imprime:

Hello World!

std::string (cont'd)

- Esta clase posee distintas funciones que le permiten realizar distintas tareas:
  - s.at(i) / s[i]: permite acceder el carácter i-esimo del string 's'
  - s1 == s2: compara el contenido de los dos strings 's1' y 's2'.
  - s.c_str(): retorna un puntero a su primer elemento. Esta este puntero permite utilizar el string 's' como un array de chars, es útil cuando una función requiere que se le pase un puntero a char y no un string.
  - s.substr(x, y): retorna un *std:string* de 's' que comienza en la posición x y tiene una longitud y.
  - s.find(char|string): busca un substring en 's'. Si se encuentra, retorna la primer posición de la primer ocurrencia de la substring. Si no encuentra la substring, se retorna el valor *std::string::npos*.

Enum y Enum Class

- Los *enum* funcionan en forma similar al lenguaje C.
- Con *class*, se evitan conflictos con los nombres al utilizar la referencia.
- El siguiente código imprime los valores 8 (mes de julio) y 2 (nombre Julio).

```
#include <iostream>

int main(){
   enum class Meses {ENERO=2, FEBRERO, MARZO, ABRIL, MAYO, JUNIO, JULIO,
            AGOSTO, SEPTIEMBRE, OCTUBRE, NOVIEMBRE, DICIEMBRE};
   // Si Nombres no fuera class, habria conflicto por JULIO
   enum class Nombres {CERGIO, JUAN, JULIO};

   std::cout << static_cast<int>(Meses::JULIO) << std::endl;
   std::cout << static_cast<int>(Nombres::JULIO) << std::endl;
   //std::cout << Meses::MARZO << std::endl; // ESTO PRODUCE UN ERROR!
   return 0;
}
```

Deducción Automática de Tipos

- C++ puede deducir en forma automática el tipo de variable usando *auto*.
- Sólo se puede usar *auto* para inferir un tipo en tiempo de compilación.
- Hay posibilidades que si se mezclan variables (por ejemplo, int y char), el resultado puede ser ambiguo.

```
#include <iostream>
using namespace std;

int func1 (){return 1;}

double func2 (){return 2.5;}

int main(){
   auto a = 1, b = 2, c = 3;
   auto d = 4, e = 5;
   auto f = '7';          // ASCII Dec = 55
   auto g = a+b+c+d+e+f;
   cout << "La variable a es tipo: " << typeid(a).name() << endl;
   cout << "La variable f es tipo: " << typeid(f).name() << endl;
   cout << "La variable g es tipo: " << typeid(g).name() << endl;
   cout << "El valor de g es: " << g << endl;

   // El resultado puede no ser el esperado
   auto result = func2() + func1();

   cout << "Result es tipo: " << typeid(result).name() << endl;
   cout << "El valor de result es: " << result << endl;

   return 0;
}
```

Operadores

- En C++ hay diferentes tipos de operadores:

1. Operador de asignación (=) y comparación (==)
2. Operador aritmético (operador binario):

   +, -, *, /, %: suma, resta, multiplicación, división y módulo,
3. Operador de bits (operadores unarios y binario):

   &, |, ^, <<, >>, ~: AND, OR, XOR, shift izquierda, shift derecha y negación.

   Cual es la diferencia entre & y &&?
4. Operador compuesto de asignación (operador binario):

   +=, -=, *=, /=, %=, &=, |=, ^=: realiza la operación y asigna el resultado.

Operadores

5. Operadores de incremento y decremento (operador unario):

   ++i, --i: pre incremento/decremento, suma 1 y luego hace operaciones.

   i++, i--: post incremento/decremento, hace operaciones y luego suma 1.

Ejemplo:

```
#include <iostream>
int main(){
    int x = 1, y = 1;
    std::cout << x++ * y++ << std::endl;        // imprime 1 (1*1), pero serán x = 2 e y = 2
    std::cout << x++ * ++y << std::endl;        // imprime 6 (2*3), pero serán x = 3 e y = 3
    std::cout << --x * y++ << std::endl;        // imprime 6 (2*3), pero serán x = 2 e y = 4
    std::cout << x-- * --y << std::endl;        // imprime 6 (2*3), pero serán x = 1 e y = 3
    std::cout << x * y << std::endl;            // imprime 3 (1*3), pues serán x = 1 e y = 3

    return 0;
}
```

Sentencia Condicional

- El operador ternario: ***Condición ? Expresión 1 : Expresión 2***

Si la condición es verdadera, la expresión 1 es ejecutada, sino, la expresión 2.

- La condición a verificar se escribe mediante los operadores lógicos.

Operador	Operación
&&	AND
||	OR
!	Negación
<	Menor que
>	Mayor que
==	Igual a
<=	Menor o igual a
>=	Mayor o igual a
!=	No es igual a

Operador	Asociatividad
( )	Izquierda a derecha
++, --, !, +, -, (cast), sizeof	Derecha a izquierda
*, /, %	Izquierda a derecha
+, -	Izquierda a derecha
<, <=, >, >=	Izquierda a derecha
==, !=	Izquierda a derecha
&&	Izquierda a derecha
||	Izquierda a derecha
? :	Derecha a izquierda
=, +=, -=, *=, /=	Derecha a izquierda

Sentencia Condicional – if / else

- Las sentencias if/else se utilizan como en el caso de C.
- Los casos de if/else pueden ser apilados y/o anidados:

if/else apilados:

```
if (condicion1){
    ...;
} else {
    ...; }
if (condicion2){
    ...;
} else {
    ...;
}
```

If/else anidados:

```
if (condicion1){
    if (condicion2){
        ...;
    else
        ...;
}
else {
    ...;
}
```

La sentencia *switch*

- El switch también es utilizado en forma similar a C, solo que en C++ se pueden utilizar también los *enum class.*
- Permite evitar problemas de nombres COLORES::ROJO.
- Permite tipear los datos en el enum: **short int**

```
#include <iostream>
using namespace std;
enum class COLORES: short int { ROJO, VERDE, AZUL };

int main() {
   COLORES colorAuto = COLORES::ROJO;

   switch (colorAuto) {
      case COLORES::ROJO:
         cout << "El auto es rojo." << endl << endl; break;
      case COLORES::VERDE:
         cout << "El auto es verde." << endl << endl; break;
      case COLORES::AZUL:
         cout << "El auto es azul." << endl << endl; break;
      default:
         cout << "No es un color contemplado." << endl;
   }
   return 0;
}
```

El código imprime:

El auto es rojo.

Sentencias de Iteración - *while* y *do/while*

- Repiten una porción de código y se detiene cuando la condición se cumple (si la terminación no se fuerza).
- ¿Cuál es la diferencia?

```
sentencia1 ;
sentencia2 ;

while (condición){

    sentencia3;

    ⋮

    sentenciaN;

}
```

```
sentencia1 ;
sentencia2 ;

do {

    sentencia3;

    ⋮

    sentenciaN;

} while (condición)
```

Sentencias de Iteración - *while* y *do/while*

- Repiten una porción de código y se detiene cuando la condición se cumple (si la terminación no se fuerza).
- Si la condición no se cumple, en el do/while, se ejecuta el contenido entre llaves sólo una vez, mientras que en el while, no se ejecuta nunca.

```
sentencia1 ;
sentencia2 ;

while (condición){

    sentencia3;

    ⋮

    sentenciaN;

}
```

```
sentencia1 ;
sentencia2 ;

do {

    sentencia3;

    ⋮

    sentenciaN;

} while (condición)
```

Sentencias de Iteración - *for*

- El *for* funciona en forma similar a como funciona en el lenguaje C.
- Las expresiones 1 y 3 permiten definir y modificar una variable que se puede evaluar en la condición para continuar la iteración.

```
sentencia1 ;
sentencia2 ;
for (expresión1; condición; expresión3){
    sentencia3;
    ⋮
    sentenciaN;
}
```

```
int N = 10;
for (int i=0; i<N; i++){
    std::cout << i << std::endl;
}
```

El *for basado en rango*

- Este *for permite iterar sobre rangos de variables como ser un array, un std::vector, o algún otro container*.
- Usualmente el tipo de elemento se define como auto y lo que recorre es el contenido del contenedor.

```
<definición de container>
for (tipo de elemento : container){
    sentencia1;
    ⋮
    sentenciaN;
}
```

El *for basado en rango* - *Ejemplos*

```
#include <iostream>
#include <array>

int main(){
        std::array<int, 6> v = { 0, 1, 2, 3, 4, 5 };  // definición de std::array
        int a[] = {0, 1, 2, 3, 4, 5};

        for ( const int& i : v )            // accede por referencia constante el valor entero
                std::cout << i << " ";
        std::cout << "\n";

        for ( auto i : v )                  // accede por valor, el tipo de i es int
                std::cout << i << " ";
        std::cout << "\n";

        for ( auto n : a )                  // para el array 'a'
                std::cout << n << " ";

        return 0;
}
```

Las sentencias *break* y *continue*

- La sentencia ***break*** interrumpe la iteración saliendo del ciclo de iteraciones.
- La sentencia ***continue*** interrumpe la iteración actual, pero continua con el ciclo de iteraciones.

```
#include <iostream>

int main(){
    int N = 10;
    for (int i = 0; i<N; i++){
        if (i==5){
            std::cout << "Dentro del if" << std::endl;
            break/continue;
        }
        std::cout << "El valor de i es: " << i << std::endl;
    }
    return 0;
}
```

Resultado con ***continue***

```
El valor de i es: 0
El valor de i es: 1
El valor de i es: 2
El valor de i es: 3
El valor de i es: 4
Dentro del if
El valor de i es: 6
El valor de i es: 7
El valor de i es: 8
El valor de i es: 9
```

Resultado con ***break***

```
El valor de i es: 0
El valor de i es: 1
El valor de i es: 2
El valor de i es: 3
El valor de i es: 4
Dentro del if
<sale del for>
```

Try / Catch y Throw

Los comandos *try* y *catch* funcionan juntos para el manejo de excepciones. El bloque try define un scope en el cual, si se ejecuta un throw, la ejecución normal se interrumpe y se transfiere el control al catch correspondiente. Es decir, se abandona el scope del try y se ingresa al del catch.

Cuando se ejecuta un throw, el bloque donde ocurre deja de ejecutarse y comienza la búsqueda de un catch compatible. Si no existe un catch adecuado, el programa termina su ejecución.

```
try {
   // Codigo que puede llegar a arrojar una excepcion
} catch (exception_type e) {
   // Maneja la excepción de tipo exception_type
} catch (...) {
   // Maneja cualquier tipo de excepción (catch-all)
}
```

La excepción por *throw+mensaje*, es un tipo texto dado por una cadena de caracteres.
El “catch (…)” también funciona con un throw, pero se usa cuando no es uno específico.

```
#include <iostream>

int main() {
   try {
      int x = 10, y = 0;
      if (y == 0)   // Arroja un error con el texto dado.
         throw "Division por zero!";
      int result = x / y;
   } catch (const char* e) {  // Captura la excepcion
      std::cout << "Error: " << e << std::endl;
   }  // Imprime el error con el texto dado en throw.

   std::cout << "El program continua..." << std::endl;
   return 0;
}
```

Try / Catch y Throw (cont'd)

- En una sección del código pueden producirse distintos tipos de errores.
- Para ello, se pueden definir múltiples bloques catch, cada uno asociado a un tipo de excepción distinto.
- Los bloques catch se evalúan en orden, y el primero cuyo tipo sea compatible con la excepción lanzada es el que la captura.
- Si la excepción no coincide con ninguno de los tipos especificados, se ejecuta el bloque catch(...), que actúa como manejador genérico.

```
#include <iostream>
#include <stdexcept>  // Para atrapar el std::runtime_error

int main() {
   try {
      throw std::runtime_error("Algo salió mal!");
      //throw "Algo salió muy mal!";
      //throw std::logic_error("Una cuenta salió muy mal!");
   } catch (const std::runtime_error& e) { // Objeto con &
      std::cout << "Runtime error: " << e.what() << std::endl;
   } catch (const char* e) { // cadena de caracteres char*
      std::cout << "Error: " << e << std::endl;
   } catch (...) {  // cualquier otro tipo de retorno de la excepción
      std::cout << "Arrojó una excepción desconocida" << std::endl;
   }
   return 0;
}
```

Funciones

- Las funciones permiten dividir el código en tareas, obteniendo así:
  - Un código más fácil de leer y seguir,
  - Reutilizar código,
  - Reducir el mantenimiento del código al evitar código repetido.
- La definición de una función es la siguiente:

```
tipo_a_retornar nombre(dataType argumento,…){
    sentencia1;
    ⋮
    sentenciaN;
    return myVar;
}
```

Funciones

- Al igual que en C, las funciones en C++ poseen tres partes:
  1) El prototipo de la función,
  2) La llamada a la función,
  3) La definición de la función.

- Las funciones pueden tener valores default en sus parámetros (esto es una funcionalidad de C++).

- En el ejemplo, se definen los valores default como cero, para evitar entrar en el *for* y retornar uno ($0^0 = 1$, por definición).

```
#include <iostream>

1) int nPower(int val=0, int exp=0);

int main(){
2)  std::cout << nPower(3,4) << std::endl;
    return 0;
}

3) int nPower(int val=0, int exp=0){
    int valPow = 1;
    for (int i=0; i<exp; i++)
        valPow *= val;
    return valPow;
}
```

Funciones - Recursión

- Funciones recursivas son funciones que se llaman a sí mismas.
- El concepto es reducir un problema en problemas más pequeños y similares al problema original.
- Cada llamada a la función utiliza recursos, pudiendo generar problemas de performance.
- Funciones recursivas pueden ser optimizadas mediante memoización.
- Toda función recursiva debe tener un caso base que le permita terminar la recursividad y defina el caso más elemental.

```
#include <iostream>

int nPower(int base, int exponent);

int main() {
  int x = 3, exp = 4;
  std::cout << nPower(x, exp) << std::endl;
  return 0;
}

int nPower(int val, int exp) {
  if (exp == 0) {
    return 1;      // Case base: x^0 = 1
  } else {
    return val * nPower(val, exp - 1);
  }
}
```

La palabra reservada *inline*

- Hace el prototipo y la definición de la función en una forma especial, esto es:

  *inline float cube(float val) {return val*val*val;}*

- La palabra *inline* avisa al compilador que las llamadas a la función (en este caso *cube*) <u>puede</u> ser reemplazada por el contenido de la función.
- Usar *inline* no garantiza que el compilador haga el reemplazo, esto depende del largo de la función y del nivel de optimización requerido al compilar.
- <u>Ventaja</u>: dado que varias tareas se realizan al llamar una función (guardar contexto, saltar a la función, retornar, etc), el programa puede ser más rápido.
- <u>Desventaja</u>: el programa puede ser más grande (más código repetido).
- Se recomienda utilizarlo con funciones cortas que se usan frecuentemente.
- <u>Ventaja frente a macros</u>: type-safe, más fácil para el debugging y para el mantenimiento de código.

Sobrecarga de Funciones

- Sobrecarga de funciones es tener funciones con los mismos nombres, pero con distintos tipos de parámetros (funcionalidad de C++).
- El compilador es el encargado de realizar la llamada a la función correspondiente en según el tipo de dato del parámetro.
- La sobrecarga de funciones hace más legible el código al evitar tener distintos nombres de funciones para la misma tarea.

El código imprime: 27
166.375

```
#include <iostream>

int cube(int x);
double cube(double y);

int main() {
  int x = 3 ;
  double y = 5.5;
  std::cout << cube(x) << std::endl;
  std::cout << cube(y) << std::endl;

  return 0;
}

int cube(int x) { return x*x*x ; }
double cube(double y) { return y*y*y; }
```

El calificador *const*

- Si se desea obtener un valor constante se utiliza la palabra reservada const:

```
const int x = 1;
```

- Con una constante, se puede llegar a definir el largo de un *std::array*:

```
const int N = 5;
std::array<int, N> arr = {1, 2, 3, 4, 5};
```

- Una vez declarada la constante, la misma debe ser inicializada y no se podrá modificar. El intento de modificar una constante o no inicializarla resulta en un error en tiempo de compilación:

```
const int N1 = 5;
const int N2;          // error en tiempo de compilación
N1++;                  // error en tiempo de compilación
```

El calificador *const con funciones*

- El calificador *const* también puede ser usado en la lista de parámetros de una función.
- Si el parámetro nunca se altera en la función, es buena práctica el hacerlo constante (*const* también puede ser utilizado con referencias y punteros).
- Se puede usar *const* en el valor a retornar, pero realmente no tiene mucha diferencia.

```
const float cube(const float x) {return x*x*x;}

int main(){
  float x = 3.3;
  const float f1 = cube(x);
  float f2 = cube(x);
  …
  return 0;
}
```

El calificador *constexpr*

- El calificador *constexpr* (C++11 pero la mayoría de las funcionalidades aparecieron en C++14) se utiliza para indicar que se define una constante que puede ser evaluada en tiempo de compilación.

- Tiene distintos usos, en el ejemplo, se resuelve recursivamente Fibonacci del 0 al 9 para completar la tabla.

- De esta manera, se tienen estos primeros 10 elementos calculados durante el tiempo de compilación, mejorando la performance del programa.

```
#include <iostream>

constexpr int fib(int n) {
   if (n == 0) return 0;
   if (n == 1) return 1;
   return fib(n - 1) + fib(n - 2);
}

constexpr int fibonacci[10] = {
   fib(0), fib(1), fib(2), fib(3), fib(4), fib(5),
   fib(6), fib(7), fib(8), fib(9)
};

int main() {
   for (int i = 0; i < 10; ++i) {
      std::cout << fibonacci[i] << " ";
   }
   std::cout << std::endl;
}
```

El calificador *constexpr*

- Al utilizar *constexpr,* es el compilador quien decide de realizar la evaluación en tiempo de ejecución.
- En el código de arriba, la dimensión de *arr* es obtenida en tiempo de compilación.
- En el código de abajo, dado que la variable *n* es declarada como 3 en tiempo de ejecución, la dimensión del array *arr* es evaluada en tiempo de ejecución.

Evaluado en tiempo de compilación

```
constexpr int cube(int x){
  return x*x*x;
}

int main() {
  int arr[cube(3)];
}
```

Evaluado en tiempo de ejecución

```
constexpr int cube(int x){
  return x*x*x;
}

int main() {
  int n = 3;
  int n3 = cube(n);
}
```

El calificador *constexpr* y *consteval*

- En la versión C++20, la funcionalidad de *constexp* fue expandida y se incluyó el calificador *consteval*.
- Con *consteval*, la función debe ser evaluada en tiempo de compilación.
- Si *multiplicar* recibe un parámetro definido como variable (la variable entera *n*), se tendrá un error de compilación debido a que no es constante.

```
constexpr int sumar(int x, int y) {
   return x + y;
}

consteval int multiplicar(int x, int y) {
   return x * y;
}

int main() {
   int n = 3;

// Evaluado en tiempo de compilación o ejecución.
   int a = sumar(2, 3);

// Debe ser evaluado en tiempo de compilación.
   int b = multiplicar(2, 3);
}
```

Archivos

El tema de archivos en C++ se introducirá brevemente en esta sección pero se verá con mayor detalle posteriormente.

Inicialmente, en C++ se hace uso de dos librerías, una para leer, *ifstream*, y otra para escribir, *ofstream*, a archivos.

Los archivos pueden leerse o escribirse en forma de texto o binario de acuerdo a los flags que se indiquen.

Indicación	Descripción
std::ios::in	Lectura (el default de *ifstream*)
std::ios::out	Escritura (el default de o*fstream*)
std::ios::binary	Modo binario (el modo default es texto)
std::ios::app	Modo *Append* (agrega datos al final del archivo)
std::ios::ate	Abre el archivo e ubica el cursor al final del mismo (lectura y escritura)
std::ios::trunc	Borra el contenido del archivo a escribir.

Ejemplo – Archivos de Texto

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
   // Abre el archivo en modo texto para leer
   std::ifstream inFile(“ArchivoTest.txt");
   if (inFile.is_open()) {
      std::string line;
      while (getline(inFile, line)) // Lee línea por línea
         std::cout << line << '\n’; // Texto en pantalla
      inFile.close(); // Cierra el archivo
   } else
      std::cerr << "Error el archivo no esta abierto!\n";
   // No es necesario cerrar el archivo manualmente,
   //pero esto hace que se guarden los datos.
   return 0;
}
```

Leer archivo de texto

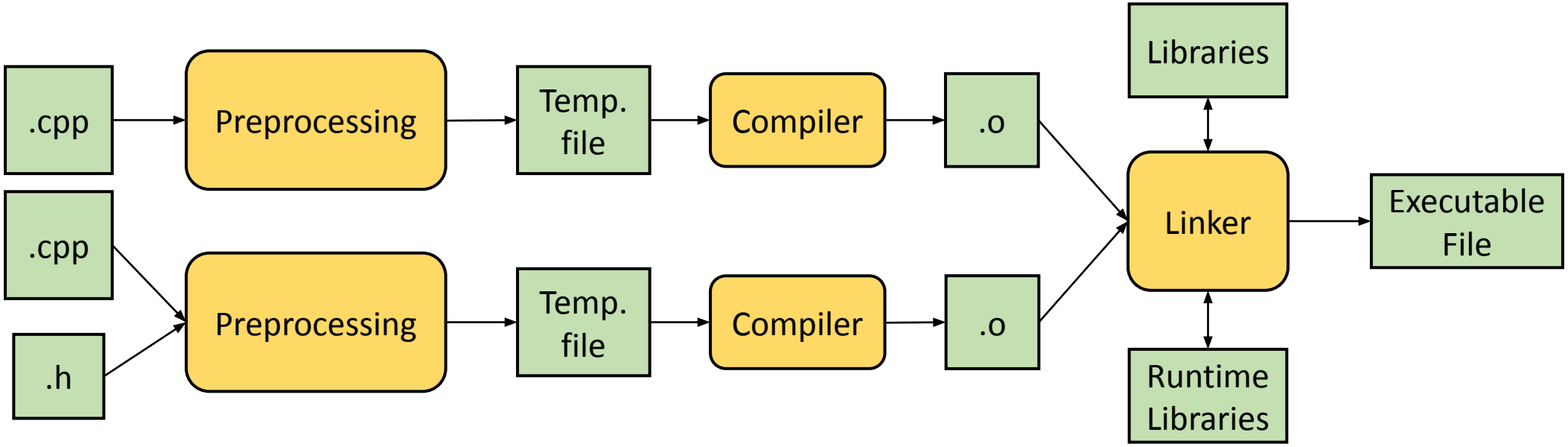
```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ofstream outFile(“ArchivoTest.txt");
   if (outFile.is_open()) { // Escribe por línea de texto
      outFile << "Hola, bienvenidos a I102 PdeP!\n";
      outFile << "Escribir a un archivo de texto es facil.\n";
      outFile.close(); // Cierra el archivo
      cout << "Archivo creado.\n";
   } else
      cerr << "Error abriendo el archivo!\n";
   // No es necesario cerrar el archivo manualmente,
   //pero esto hace que se guarden los datos.
   return 0;
}
```

Escribir archivo de texto

Comandos del Preprocesador

- Los comandos del preprocesador son comandos que se ejecutan en la etapa de preprocesamiento y generan el código a compilar.

- Estos comandos se encuentran dentro de los archivos que ingresan al preprocesador.



#include

Este comando permite incluir el contenido del archivo indicado, el cual puede ser un standard de C/C++ o un archivo definido por el usuario. Prácticamente se puede considerar que el comando es reemplazado con el contenido del archivo.

#include <file>

Cuando se utilizan los símbolos “<>”, el preprocesador buscará los archivos en un directorio predeterminado (standard include directories) para acceder los header solicitados.

#include “file.h”

Cuando se utilizan comillas, el preprocesador buscará el archivo header a incluir en el directorio del proyecto. No obstante, se puede cambiar la ubicación del archivo agregándole el path.

Para ambos casos, si no encuentra el archivo, lo buscará en el path indicado con el flag de compilación –I.

#define

Este comando permite definir macros y constante que son reemplazados en tiempo de compilación:

#define nombreMacro textoAReemplazar

Ejemplos:

```
#define PI 3.14159              // Reemplazará la palabra PI por 3.14159
#define MIN(a,b)  (  ( (a) < (b) )  ?  a : b  )     // Macro para encontrar mínimo
```

En ambos casos el *textoAReemplazar* substituye el texto nombre del macro. En el caso de MIN(a,b) este se reemplaza en tiempo de preprocesado pero se evalúa en tiempo de ejecución o compilación.

#undef

- Este comando se utiliza para cancelar la definición de una macro previamente definida mediante *#define*.

- En caso de que el comando *#undef* se utilice con una macro que no exista, este será ignorado y no se observará un error.

```
#include <iostream>
#define PI 3.14159
#undef PI

int main () {  // Error, PI no esta definido.

    std::cout << "Value of PI: " << PI << std::endl;

    return 0;
}
```

Instrucciones Condicionales

- *#if, #elif, #else* y *#endif* - Permite incluir o excluir código basado en la condición provista:

```
#define FEATURE_1 1                // El feature 1 fue implementado
#define TESTS_FEATURE_1 1          // El feature 1 paso las pruebas
#define FEATURE_2 1                // El feature 2 fue implementado
#define TESTS_FEATURE_2 0          // El feature 2 no paso las pruebas

#if FEATURE_1 && TESTS_FEATURE_1
    std::cout << “El feature 1 esta habilitado" << std::endl;
#else
    std::cout << “El feature 1 esta deshabilitado" << std::endl;
#endif

#if !TESTS_FEATURE_2
    std::cout << “El feature 2 no pasó las pruebas " << std::endl;
#elif FEATURE_2
    std::cout << “Feature 2 esta habilitado para su instalación" << std::endl;
#endif
```

Instrucciones Condicionales (cont'd)

- *#ifdef, #ifndef, #elifdef* (C++23) *y #elifndef* (C++23) - Permite incluir o excluir código basado en que un macro esté (#ifdef) o no esté (#ifndef – if not defined) definido:

```
#define MIN_VELOC 100
#define MAX_VELOC 200

#ifdef MIN_VELOC
	std::cout << "Hay una velocidad mínima" << std::endl;
#elifndef MAX_VELOC
	std::cout << "No hay limites de velocidad" << std::endl;
#endif
```

```
#define MIN_VELOC 100
#define MAX_VELOC 200

#ifndef MAX_VELOC
	#define MAX_VELOC 150
#elifdef MIN_VELOC
	#undef MIN_VELOC
	#def MIN_VELOC 120

#enfif
// Si no está definida MAX_VELOC definirla. Si está definida eliminar definición de MIN_VEL y definirla nuevamente como 120.
```

#error

Este comando se utiliza para generar un error de compilación con un mensaje definido por el programador.

```
#define MAIN_FEATURE 1

#if MAIN_FEATURE
	std::cout << “El feature principal esta habilitado" << std::endl;
#else
	#error “El feature principal no esta habilitado para las pruebas”
#endif
```

#line

Se utiliza para modificar el número de línea y el nombre del archivo que reporta el preprocesador y el compilador.

Se utiliza con código generado automáticamente para poder facilitar su debugging.

```
void my_funct() {
   ⋮   // líneas de código generado automáticamente

   #line 1000 "generated_code.cpp“
   <línea de código con error>

    ⋮   // líneas de código generado automaticamente
}

int main() {
   my_funct();
   return 0;
}
```

#pragma

Esta es una directiva que se usa para proveer instrucciones especiales o específicas al compilador.

Indican al compilador el generar una advertencia, realizar optimización, etc.

El comando más utilizado debido a que es soportado por múltiples compiladores, es:

#pragma once

Se utiliza para asegurar que un determinado archivo header sea incluido una única vez en la compilación.

Este comando evita que ocurran problemas por múltiples inclusiones del mismo header file.

Los Operadores # y

- El operador # remplaza el texto con el mismo texto pero entre comillas.
- Mientras que el operador ## es utilizado para concatenar dos tokens.

```
#include <iostream>
#define QUOTMRK( x ) #x

int main () {
    std::cout << QUOTMRK(HOLA C++) << std::endl;
    return 0;
}
```

Resulta en la línea de código:

```
std::cout << "HOLA C++" << std::endl;
```

```
#include <iostream>
#define concat(a, b) a ## b

int main() {
    int xy = 100;
    std::cout << concat(x, y) << std::endl;
    return 0;
}
```

Resulta en la línea de código:

```
std::cout << xy << std::endl;
```

Preguntas?